Pairwise and MSA Tool

CODE:

```
import tkinter as tk

from tkinter import ttk, messagebox



# ALIGNMENT LOGIC



# Simplified BLOSUM62 similar pairs for protein

BLOSUM62_SIMILAR = {

    ('A','S'),('A','T'),('R','K'),('N','D'),('N','S'),('D','E'),

    ('Q','E'),('Q','K'),('Q','R'),('H','Y'),('I','L'),('I','V'),

    ('L','M'),('L','V'),('K','R'),('F','Y'),('F','W'),('Y','W'),

    ('S','T'),('V','I'),('V','L'),('M','I'),('M','V'),('E','D'),

}
```

```python
GAP_OPEN   = -2

GAP_EXTEND = -1



VALID = {

    'DNA':     set('ATCG'),

    'RNA':     set('AUCG'),

    'PROTEIN': set('ACDEFGHIKLMNPQRSTVWY')

}



def detect_type(sequence):

    s = set(sequence.upper())

    if s <= VALID['DNA'] and 'T' in s:

        return 'DNA'

    elif s <= VALID['RNA'] and 'U' in s:

        return 'RNA'

    elif s <= VALID['PROTEIN']:

        return 'PROTEIN'

    return None



def score_pair(a, b, seq_type):

    if a == '-' or b == '-':

        return GAP_EXTEND

    if a == b:
```

```
        return 1

    if seq_type == 'PROTEIN':

        pair = (min(a,b), max(a,b))

        return 0 if pair in BLOSUM62_SIMILAR else -1

    return -1


def needleman_wunsch(seq1, seq2, seq_type):

    n, m = len(seq1), len(seq2)

    dp        = [[0]*(m+1) for _ in range(n+1)]

    traceback = [[None]*(m+1) for _ in range(n+1)]


    for i in range(1, n+1):

        dp[i][0]        = GAP_OPEN + (i-1)*GAP_EXTEND

        traceback[i][0] = 'UP'

    for j in range(1, m+1):

        dp[0][j]        = GAP_OPEN + (j-1)*GAP_EXTEND

        traceback[0][j] = 'LEFT'


    for i in range(1, n+1):

        for j in range(1, m+1):

            diag = dp[i-1][j-1] + score_pair(seq1[i-1], seq2[j-1],
seq_type)

            up   = dp[i-1][j]   + GAP_EXTEND

            left = dp[i][j-1]   + GAP_EXTEND
```

```
            best = max(diag, up, left)

            dp[i][j] = best

            if best == diag:

                traceback[i][j] = 'DIAG'

            elif best == up:

                traceback[i][j] = 'UP'

            else:

                traceback[i][j] = 'LEFT'


    aligned1, aligned2 = '', ''

    i, j = n, m

    while i > 0 or j > 0:

        d = traceback[i][j]

        if d == 'DIAG':

            aligned1 = seq1[i-1] + aligned1

            aligned2 = seq2[j-1] + aligned2

            i -= 1; j -= 1

        elif d == 'UP':

            aligned1 = seq1[i-1] + aligned1

            aligned2 = '-'       + aligned2

            i -= 1

        else:

            aligned1 = '-'       + aligned1
```

```
            aligned2 = seq2[j-1] + aligned2

            j -= 1


    return aligned1, aligned2


def build_midline(a1, a2, seq_type):

    mid = ''

    for a, b in zip(a1, a2):

        if a == b and a != '-':

            mid += '|'

        elif a != '-' and b != '-':

            pair = (min(a,b), max(a,b))

            mid += ':' if seq_type == 'PROTEIN' and pair in
BLOSUM62_SIMILAR else '.'

        else:

            mid += ' '

    return mid


def calc_identity(a1, a2):

    positions = [(a,b) for a,b in zip(a1,a2) if not (a=='-' and b=='-')]

    if not positions:

        return 0.0

    matches = sum(1 for a,b in positions if a==b)

    return round(matches/len(positions)*100, 2)
```

```python
def calc_similarity(a1, a2, seq_type):
    positions = [(a,b) for a,b in zip(a1,a2) if not (a=='-' and b=='-')]
    if not positions:
        return 0.0
    sim = 0
    for a,b in positions:
        if a == b:
            sim += 1
        elif seq_type == 'PROTEIN' and a != '-' and b != '-':
            if (min(a,b), max(a,b)) in BLOSUM62_SIMILAR:
                sim += 1
    return round(sim/len(positions)*100, 2)


def run_msa(sequences, names, seq_type):
    """Progressive MSA: align closest pair first, then add remaining."""
    n    = len(sequences)
    seqs = [s.upper() for s in sequences]

    # Compute all pairwise identity scores
    score_matrix = [[0.0]*n for _ in range(n)]
    for i in range(n):
        for j in range(i+1, n):
```

```
            a1, a2 = needleman_wunsch(seqs[i], seqs[j], seq_type)
            score_matrix[i][j] = score_matrix[j][i] = calc_identity(a1,
a2)


    # Find best starting pair
    best_i, best_j, best_score = 0, 1, -1
    for i in range(n):
        for j in range(i+1, n):
            if score_matrix[i][j] > best_score:
                best_score = score_matrix[i][j]
                best_i, best_j = i, j


    # Align best pair
    a1, a2 = needleman_wunsch(seqs[best_i], seqs[best_j], seq_type)
    msa = {best_i: a1, best_j: a2}
    remaining = [k for k in range(n) if k not in (best_i, best_j)]


    # Progressively add remaining sequences
    for k in remaining:
        ref = msa[best_i]
        a_ref, a_new = needleman_wunsch(ref, seqs[k], seq_type)
        gap_positions = [idx for idx, c in enumerate(a_ref) if c == '-']
        for key in msa:
            s = msa[key]
```

```
            for pos in sorted(gap_positions):
                s = s[:pos] + '-' + s[pos:]
            msa[key] = s
        msa[k] = a_new

    return msa, score_matrix

# THEME
BG         = "#0d1117"
SURFACE    = "#161b22"
SURFACE2   = "#1c2128"
BORDER     = "#30363d"
ACCENT     = "#58a6ff"
ACCENT2    = "#3fb950"
ERR        = "#f85149"
MUTED      = "#8b949e"
TEXT       = "#e6edf3"
TEXT_DIM   = "#c9d1d9"
MATCH_COL  = "#3fb950"
SIM_COL    = "#ffa657"
GAP_COL    = "#f85149"
FONT_MONO  = ("Courier New", 10)
FONT_LABEL = ("Courier New", 10, "bold")
```

```python
FONT_TITLE = ("Courier New", 14, "bold")

FONT_SMALL = ("Courier New", 9)



# GUI



class AlignmentApp:

    def __init__(self, root):

        self.root = root

        self.root.title("Sequence Alignment Tool")

        self.root.configure(bg=BG)

        self.root.geometry("1100x800")

        self.root.resizable(True, True)


        self.seq_entries = []   # list of (name_entry, seq_text) pairs

        self._build_ui()


    # Build UI

    def _build_ui(self):

        # Title

        title_frame = tk.Frame(self.root, bg=BG, pady=16)

        title_frame.pack(fill='x', padx=24)

        tk.Label(title_frame, text="⬡  SEQUENCE ALIGNMENT TOOL",

                 font=FONT_TITLE, fg=ACCENT, bg=BG).pack(side='left')
```

```
        tk.Label(title_frame, text="Needleman-Wunsch  |  Pairwise & MSA",
                 font=FONT_SMALL, fg=MUTED, bg=BG).pack(side='right',
pady=6)


        tk.Frame(self.root, bg=BORDER, height=1).pack(fill='x', padx=24)


        main = tk.Frame(self.root, bg=BG)
        main.pack(fill='both', expand=True, padx=24, pady=16)


        self._build_left(main)
        self._build_right(main)


    # Left panel
    def _build_left(self, parent):
        self.left = tk.Frame(parent, bg=SURFACE,
                             highlightbackground=BORDER,
highlightthickness=1)
        self.left.pack(side='left', fill='y', ipadx=12, ipady=12,
padx=(0,12))


        self._section_label(self.left, "SEQUENCES")


        # Scrollable sequence entry area
        canvas_frame = tk.Frame(self.left, bg=SURFACE)
```

```
        canvas_frame.pack(fill='both', expand=True, padx=12)


        self.seq_canvas = tk.Canvas(canvas_frame, bg=SURFACE,
                                    highlightthickness=0, width=320)
        scroll = tk.Scrollbar(canvas_frame, orient='vertical',
                              command=self.seq_canvas.yview)
        self.seq_canvas.configure(yscrollcommand=scroll.set)


        scroll.pack(side='right', fill='y')
        self.seq_canvas.pack(side='left', fill='both', expand=True)


        self.seq_inner = tk.Frame(self.seq_canvas, bg=SURFACE)
        self.canvas_window = self.seq_canvas.create_window(
            (0,0), window=self.seq_inner, anchor='nw')


        self.seq_inner.bind('<Configure>', self._on_frame_configure)
        self.seq_canvas.bind('<Configure>', self._on_canvas_configure)


        # Start with 2 sequences
        self._add_sequence_row()
        self._add_sequence_row()


        # Add/Remove buttons
```

```
        btn_row = tk.Frame(self.left, bg=SURFACE)

        btn_row.pack(fill='x', padx=12, pady=(8,4))

        self._btn(btn_row, "+ Add Sequence", self._add_sequence_row,

                  SURFACE2, ACCENT).pack(side='left', padx=(0,6))

        self._btn(btn_row, "− Remove Last", self._remove_last_row,

                  SURFACE2, ERR).pack(side='left')



        tk.Frame(self.left, bg=BORDER, height=1).pack(fill='x', padx=12,
pady=8)



        # Alignment type label (auto-detected)

        tk.Label(self.left, text="Type auto-detected from sequences",

                 font=FONT_SMALL, fg=MUTED, bg=SURFACE).pack(anchor='w',
padx=12)



        tk.Frame(self.left, bg=BORDER, height=1).pack(fill='x', padx=12,
pady=8)



        # Align button

        self._btn(self.left, "▶  ALIGN", self._run_alignment,

                  ACCENT2, BG, pady=10,

                  font=("Courier New", 11, "bold")).pack(padx=12,
pady=(0,8), fill='x')



        # Clear button
```

```
        self._btn(self.left, "Clear All", self._clear_all,
                  SURFACE2, MUTED, pady=6).pack(padx=12, fill='x')

        # Status
        self.status_var = tk.StringVar(value="Ready. Enter at least 2
sequences.")
        tk.Label(self.left, textvariable=self.status_var,
                 font=FONT_SMALL, fg=MUTED, bg=SURFACE,
                 wraplength=300, justify='left').pack(padx=12, pady=8,
anchor='w')

    def _on_frame_configure(self, event):
self.seq_canvas.configure(scrollregion=self.seq_canvas.bbox('all'))

    def _on_canvas_configure(self, event):
        self.seq_canvas.itemconfig(self.canvas_window, width=event.width)

    def _add_sequence_row(self):
        idx = len(self.seq_entries) + 1
        row = tk.Frame(self.seq_inner, bg=SURFACE, pady=4)
        row.pack(fill='x', pady=(0,6))

        # Name entry
```

```
        tk.Label(row, text=f"Seq {idx} Name:", font=FONT_SMALL,
                 fg=MUTED, bg=SURFACE).pack(anchor='w')
        name_entry = tk.Entry(row, font=FONT_MONO, fg=TEXT, bg=SURFACE2,
                              insertbackground=ACCENT, relief='flat',
                              highlightbackground=BORDER,
highlightthickness=1,
                              width=35)
        name_entry.pack(fill='x', pady=(2,4))
        name_entry.insert(0, f"Sequence {idx}")


        # Sequence text
        tk.Label(row, text=f"Seq {idx}:", font=FONT_SMALL,
                 fg=MUTED, bg=SURFACE).pack(anchor='w')
        seq_text = tk.Text(row, height=3, width=35,
                           font=FONT_MONO, fg=TEXT, bg=SURFACE2,
                           insertbackground=ACCENT, relief='flat',
                           highlightbackground=BORDER,
highlightthickness=1,
                           wrap='word')
        seq_text.pack(fill='x')


        self.seq_entries.append((name_entry, seq_text))


    def _remove_last_row(self):
```

```
        if len(self.seq_entries) <= 2:

            messagebox.showwarning("Minimum", "At least 2 sequences
required.")

            return

        name_entry, seq_text = self.seq_entries.pop()

        # destroy parent row frame

        seq_text.master.destroy()



    #Right panel

    def _build_right(self, parent):

        right = tk.Frame(parent, bg=BG)

        right.pack(side='left', fill='both', expand=True)



        style = ttk.Style()

        style.theme_use('default')

        style.configure('TNotebook',     background=BG,
borderwidth=0)

        style.configure('TNotebook.Tab', background=SURFACE,
foreground=MUTED,

                        font=FONT_SMALL, padding=[12,6])

        style.map('TNotebook.Tab',

                  background=[('selected', SURFACE2)],

                  foreground=[('selected', ACCENT)])
```

```
        self.notebook = ttk.Notebook(right)
        self.notebook.pack(fill='both', expand=True)

        # Tab 1: Alignment result
        tab1 = tk.Frame(self.notebook, bg=BG)
        self.notebook.add(tab1, text='  Alignment  ')
        self.align_output = self._output_box(tab1)

        # Tab 2: Score matrix (MSA only)
        tab2 = tk.Frame(self.notebook, bg=BG)
        self.notebook.add(tab2, text='  Score Matrix  ')
        self.matrix_output = self._output_box(tab2)

        # Legend
        legend = tk.Frame(right, bg=BG)
        legend.pack(fill='x', padx=8, pady=(4,0))
        for sym, col, label in [
            ('|', MATCH_COL, 'Identical'),
            (':', SIM_COL,   'Similar (protein)'),
            ('.', MUTED,     'Mismatch'),
            (' ', ERR,       'Gap'),
        ]:
            tk.Label(legend, text=f"  {sym}  {label}",
```

```
                    font=FONT_SMALL, fg=col, bg=BG).pack(side='left')


    #Helpers

    def _section_label(self, parent, text):

        tk.Label(parent, text=text, font=("Courier New", 8, "bold"),

                 fg=MUTED, bg=SURFACE).pack(anchor='w', padx=12,
pady=(12,4))


    def _btn(self, parent, text, cmd, bg, fg, pady=6,

             font=FONT_LABEL):

        return tk.Button(parent, text=text, command=cmd,

                         font=font, fg=fg, bg=bg,

                         relief='flat', cursor='hand2',

                         activebackground=BORDER, activeforeground=TEXT,

                         padx=8, pady=pady)


    def _output_box(self, parent):

        frame = tk.Frame(parent, bg=BG)

        frame.pack(fill='both', expand=True, padx=8, pady=8)


        box = tk.Text(frame, font=FONT_MONO, fg=TEXT, bg=SURFACE,

                      relief='flat', wrap='none', state='disabled',

                      highlightbackground=BORDER, highlightthickness=1,

                      spacing3=3)
```

```
        yscroll = tk.Scrollbar(frame, orient='vertical',
command=box.yview)

        xscroll = tk.Scrollbar(frame, orient='horizontal',
command=box.xview)

        box.configure(yscrollcommand=yscroll.set,
xscrollcommand=xscroll.set)


        yscroll.pack(side='right',  fill='y')

        xscroll.pack(side='bottom', fill='x')

        box.pack(side='left', fill='both', expand=True)


        box.tag_configure('header',  foreground=ACCENT,    font=("Courier
New",10,"bold"))

        box.tag_configure('success', foreground=ACCENT2)

        box.tag_configure('error',   foreground=ERR)

        box.tag_configure('muted',   foreground=MUTED)

        box.tag_configure('label',   foreground=TEXT_DIM,  font=("Courier
New",10,"bold"))

        box.tag_configure('match',   foreground=MATCH_COL)

        box.tag_configure('sim',     foreground=SIM_COL)

        box.tag_configure('name',    foreground="#d2a8ff", font=("Courier
New",10,"bold"))

        box.tag_configure('score',   foreground=SIM_COL)

        box.tag_configure('mid',     foreground=MUTED)
```

```
        return box


    def _write(self, box, text, tag=None):
        box.configure(state='normal')
        box.insert('end', text, tag) if tag else box.insert('end', text)
        box.configure(state='disabled')


    def _clear_box(self, box):
        box.configure(state='normal')
        box.delete('1.0', 'end')
        box.configure(state='disabled')


    def _clear_all(self):
        for name_e, seq_t in self.seq_entries:
            name_e.delete(0, 'end')
            seq_t.delete('1.0', 'end')
        self._clear_box(self.align_output)
        self._clear_box(self.matrix_output)
        self.status_var.set("Ready. Enter at least 2 sequences.")


    # Run alignment
    def _run_alignment(self):
        # Collect and clean inputs
```

```
        names = []

        seqs  = []

        for name_e, seq_t in self.seq_entries:

            name = name_e.get().strip() or f"Seq {len(names)+1}"

            raw  = seq_t.get('1.0', 'end')

            seq  = ''.join(raw.split()).upper()

            if seq:

                names.append(name)

                seqs.append(seq)



        if len(seqs) < 2:

            messagebox.showwarning("Too Few", "Enter at least 2
sequences.")

            return



        # Detect type from first sequence

        seq_type = detect_type(seqs[0])

        if seq_type is None:

            messagebox.showerror("Invalid", f"Cannot detect type for:
{names[0]}")

            return



        # Check all same type

        for i, s in enumerate(seqs):
```

```
            t = detect_type(s)
            if t != seq_type:
                messagebox.showerror("Type Mismatch",
                    f"'{names[i]}' detected as {t}, expected {seq_type}.")
                return

        self._clear_box(self.align_output)
        self._clear_box(self.matrix_output)

        out  = self.align_output
        mout = self.matrix_output

        if len(seqs) == 2:
            self._do_pairwise(seqs, names, seq_type, out, mout)
        else:
            self._do_msa(seqs, names, seq_type, out, mout)

        self.notebook.select(0)

    # Pairwise
    def _do_pairwise(self, seqs, names, seq_type, out, mout):
        a1, a2  = needleman_wunsch(seqs[0], seqs[1], seq_type)
        mid     = build_midline(a1, a2, seq_type)
```

```
        identity   = calc_identity(a1, a2)

        similarity = calc_similarity(a1, a2, seq_type)



        max_name = max(len(n) for n in names)



        self._write(out, "PAIRWISE ALIGNMENT\n", 'header')

        self._write(out, f"Algorithm : Needleman-Wunsch (Global)\n",
'muted')

        self._write(out, f"Type      : {seq_type}\n\n", 'muted')



        # Print in blocks of 60 characters

        block = 60

        for start in range(0, len(a1), block):

            chunk1 = a1[start:start+block]

            chunkm = mid[start:start+block]

            chunk2 = a2[start:start+block]



            self._write(out, f"{names[0]:<{max_name+2}}", 'name')

            self._write_midline_colored(out, chunk1, chunkm, seq_type)

            self._write(out, "\n")



            self._write(out, f"{'':>{max_name+2}}", 'mid')

            self._write_midline_symbols(out, chunkm)

            self._write(out, "\n")
```

```
            self._write(out, f"{names[1]:<{max_name+2}}", 'name')
            self._write_midline_colored(out, chunk2, chunkm, seq_type)
            self._write(out, "\n\n")

        self._write(out, "—"*50 + "\n", 'muted')
        self._write(out, f"Identity   : ", 'label')
        self._write(out, f"{identity}%\n", 'score')
        self._write(out, f"Similarity : ", 'label')
        self._write(out, f"{similarity}%\n", 'score')
        self._write(out, f"Length     : ", 'label')
        self._write(out, f"{len(a1)} positions\n", 'muted')

        # Score matrix tab (just the pair)
        self._write(mout, "PAIRWISE SCORE SUMMARY\n", 'header')
        self._write(mout, "—"*40 + "\n", 'muted')
        self._write(mout, f"{names[0]} vs {names[1]}\n", 'name')
        self._write(mout, f"  Identity   : {identity}%\n", 'score')
        self._write(mout, f"  Similarity : {similarity}%\n", 'score')

        self.status_var.set(f"Pairwise done. Identity: {identity}%
Similarity: {similarity}%")

    #  MSA
```

```
    def _do_msa(self, seqs, names, seq_type, out, mout):

        msa, score_matrix = run_msa(seqs, names, seq_type)

        n = len(seqs)

        max_name = max(len(n) for n in names)



        self._write(out, "MULTIPLE SEQUENCE ALIGNMENT\n", 'header')

        self._write(out, f"Algorithm : Progressive (Needleman-Wunsch)\n",
'muted')

        self._write(out, f"Type      : {seq_type}\n", 'muted')

        self._write(out, f"Sequences : {n}\n\n", 'muted')



        # Get aligned sequences in order

        aligned = [msa[i] for i in range(n)]

        length  = len(aligned[0])



        # Print in blocks of 60

        block = 60

        for start in range(0, length, block):

            self._write(out, f"Position {start+1}-{min(start+block,
length)}\n", 'muted')

            for i, (name, aln) in enumerate(zip(names, aligned)):

                chunk = aln[start:start+block]

                self._write(out, f"{name:<{max_name+2}}", 'name')

                for c in chunk:
```

```
                    if c == '-':
                        self._write(out, c, 'error')
                    else:
                        self._write(out, c)
                self._write(out, "\n")

            # Consensus line
            cons_chunk = ''
            for pos in range(start, min(start+block, length)):
                col = [aligned[i][pos] for i in range(n)]
                if all(c == col[0] and c != '-' for c in col):
                    cons_chunk += '*'
                elif '-' in col:
                    cons_chunk += ' '
                else:
                    cons_chunk += '.'
            self._write(out, f"{'Consensus':<{max_name+2}}", 'muted')
            self._write(out, cons_chunk + "\n\n", 'match')

        self._write(out, "—"*50 + "\n", 'muted')
        self._write(out, "* = identical  . = variable  gap = -\n",
'muted')

        # Score matrix tab
```

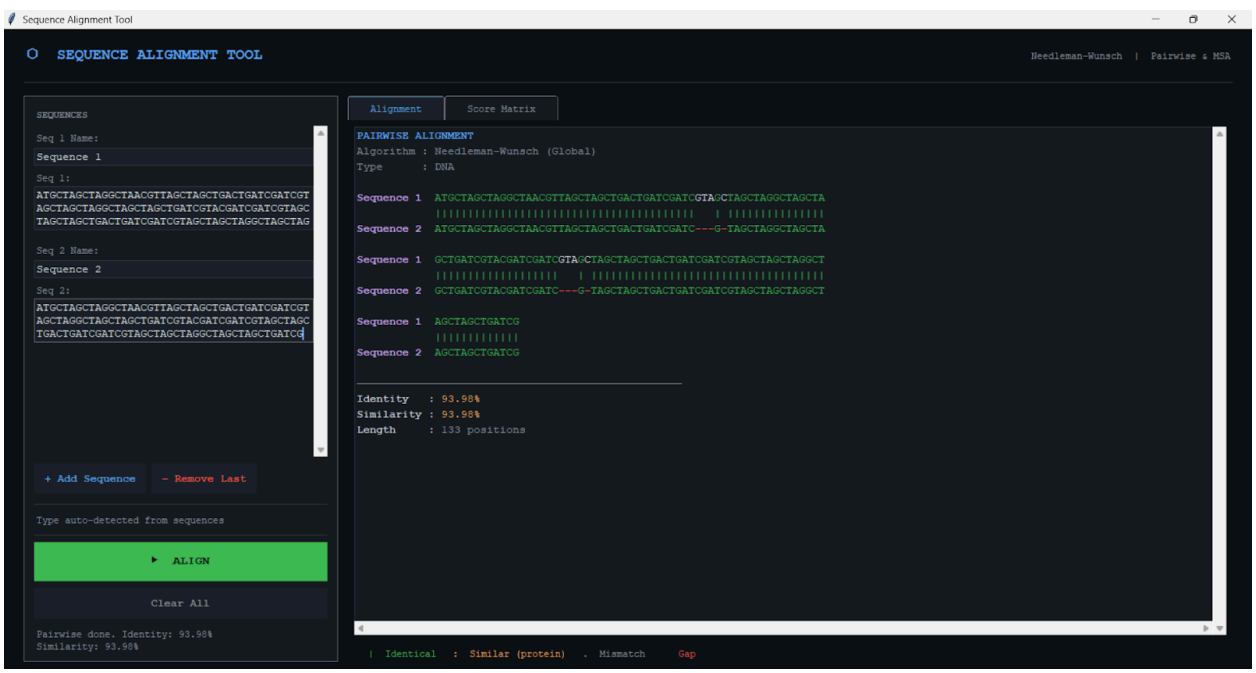
```
        self._write(mout, "PAIRWISE IDENTITY SCORE MATRIX (%)\n",
'header')

        self._write(mout, "—"*50 + "\n\n", 'muted')



        col_w = 10

        header = f"{'':>{max_name+2}}" + ''.join(f"{names[j]:>{col_w}}"
for j in range(n))

        self._write(mout, header + "\n", 'label')



        for i in range(n):

            row = f"{names[i]:<{max_name+2}}"

            for j in range(n):

                if i == j:

                    row += f"{'100.0':>{col_w}}"

                else:

                    row += f"{score_matrix[i][j]:>{col_w}}"

            self._write(mout, row + "\n", 'score')



        self._write(mout, "\nPairwise Identity & Similarity:\n\n",
'label')

        for i in range(n):

            for j in range(i+1, n):

                a1, a2 = needleman_wunsch(aligned[i], aligned[j],
seq_type)

                idn = calc_identity(a1, a2)
```

```
                sim = calc_similarity(a1, a2, seq_type)
                self._write(mout, f"  {names[i]} vs {names[j]}\n", 'name')
                self._write(mout, f"    Identity   : {idn}%\n", 'score')
                self._write(mout, f"    Similarity : {sim}%\n\n", 'score')

        self.status_var.set(f"MSA done. {n} sequences aligned. {length} positions.")

    def _write_midline_colored(self, box, seq_chunk, mid_chunk, seq_type):
        for c, m in zip(seq_chunk, mid_chunk):
            if c == '-':
                self._write(box, c, 'error')
            elif m == '|':
                self._write(box, c, 'match')
            elif m == ':':
                self._write(box, c, 'sim')
            else:
                self._write(box, c)

    def _write_midline_symbols(self, box, mid_chunk):
        for m in mid_chunk:
            if m == '|':
                self._write(box, m, 'match')
            elif m == ':':
```

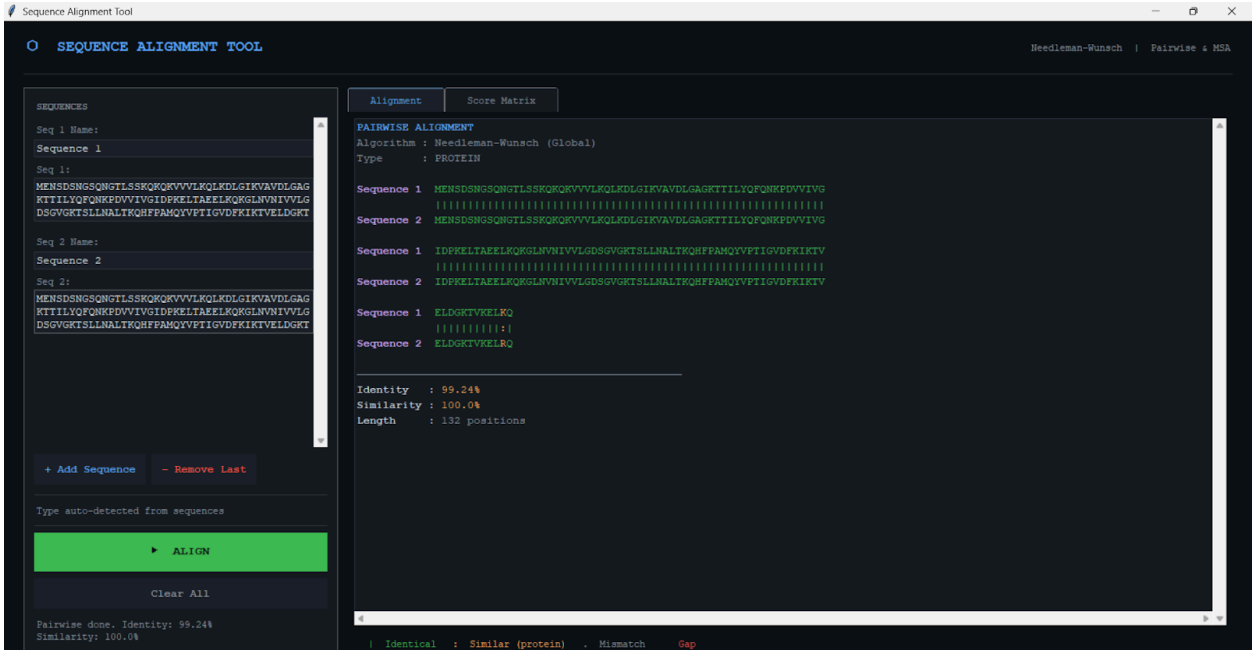
```
                self._write(box, m, 'sim')

            else:

                self._write(box, m, 'muted')




# RUN

root = tk.Tk()

app  = AlignmentApp(root)

root.mainloop()
```

SS OF TOOL:

DNA:



For protein:



Software Requirements Specification (SRS)

Sequence Alignment Tool

1. Introduction

1.1 Purpose

This software is designed to perform sequence alignment for DNA, RNA, and protein sequences. It helps users compare biological sequences and identify similarities, differences, and evolutionary relationships.

1.2 Scope

The tool provides:

- Pairwise sequence alignment (2 sequences)
- Multiple sequence alignment (MSA) (3 or more sequences)
- Identity and similarity calculation
- Visual alignment output with symbols

2. Overall Description

2.1 Product Perspective

This is a standalone desktop application built using Python (Tkinter GUI). It does not require internet access.

2.2 Product Functions

- Input multiple sequences
- Auto-detect sequence type (DNA, RNA, Protein)
- Perform alignment using Needleman-Wunsch algorithm
- Display aligned sequences with symbols:
    - | → Match
    - : → Similar (protein)
    - . → Mismatch
    - - → Gap
- Show identity and similarity percentage
- Display score matrix (for MSA)

2.3 User Characteristics

- Students (Bioinformatics / CS / Biotechnology)
- Researchers (basic level)
- No advanced technical knowledge required

3. Functional Requirements

3.1 Input Requirements

- User must enter at least 2 sequences
- Sequences must contain valid characters:
    - DNA → A, T, C, G
    - RNA → A, U, C, G
    - Protein → Amino acid letters

3.2 Processing Requirements

- Detect sequence type automatically
- Validate all sequences are of same type
- Perform:
    - Pairwise alignment (if 2 sequences)
    - MSA (if >2 sequences)

3.3 Output Requirements

- Display aligned sequences
- Show:
    - Match/mismatch/gap symbols
    - Identity percentage
    - Similarity percentage
- For MSA:
    - Show consensus line
    - Display score matrix

3.4 Error Handling

- Show error if:
    - Less than 2 sequences
    - Invalid sequence characters
    - Mixed sequence types

4. Non-Functional Requirements

4.1 Performance

- Should handle sequences up to moderate length (~1000 bases) efficiently

4.2 Usability

- Simple and user-friendly GUI
- Clear buttons and labels
- Readable output format

4.3 Reliability

- Should not crash on incorrect input
- Must show proper error messages

4.4 Portability

- Runs on any system with Python installed

5. User Interface Requirements

- Input panel for sequences
- Buttons:
    - Add Sequence
    - Remove Sequence
    - Align
    - Clear All
- Output panel:

    - Alignment tab
    - Score Matrix tab

6. Constraints

- Uses Needleman-Wunsch algorithm only
- Simplified similarity scoring (not full BLOSUM62 matrix)
- GUI built using Tkinter

7. Future Enhancements

- Add Smith-Waterman (local alignment)
- Import sequences from FASTA files
- Use full BLOSUM62 scoring matrix
- Export results to file
- Add graphical visualization